

Evaluating Streaming and In-Memory CSV Approaches for Data Ingestion and Querying

Brandon Llanes
Dept. of Engineering
San Jose State University
San Jose, California, USA
brandon.llanes@sjsu.edu

Meghana Koti
Dept. of Engineering
San Jose State University
San Jose, California, USA
meghana.koti@sjsu.edu

I.

II. INTRODUCTION

The Yellow Taxi Trip Records dataset from NYC OpenData contains all documented taxi rides taken within the corresponding year. Each year's dataset averages more than 30 million rows of 18 columns. To fit the file size data requirement, sequential year datasets from 2019 to 2022 were used to combine to a total of 24.1GB on disk. The schema pertained to date/time of the record, passenger information, the dropoff destination, distance travelled, and payment received. The datetime of pickup and dropoff are strings whereas the rest of the dataset was numerical.

With the large amount of data in CSVs, the goal was to create an application to load all of the data and perform queries to retrieve a subset. The project was split into three phases: a serial baseline, a parallel implementation, and finally a redefinition of the Taxi Trip Record model.

At a first glance, ingestion of a dataset this large would consequently result in an upfront downtime before allowing queries to be done during the first phase. Instead of just storing the data in memory, another option could be utilized: streaming. Instead of storing the data, when a query is run, the application could keep track of one record only as it traverses the files.

To see the advantages and disadvantages of data streaming versus in-memory, the two approaches were simultaneously implemented throughout the three phases. This paper serves to document the research findings of comparing the two approaches.

III. PHASE ONE - SERIAL

Phase one acts as a baseline to phase two and three. In this phase, the system is implemented entirely in serial form to focus on functionality and abstraction rather than performance optimization. The primary objectives were: define a taxi trip record model, implement a CSV reader and parser for multi-file

ingestion, develop a predicate-based query API, and build a benchmarking framework to evaluate performance.

A. In-Memory

1) *Data Ingestion Architecture*: For the in-memory approach, a CSVReader class looks through a data directory for CSV files, adding all to a vector to be read through. In combination with the TaxiTripParser class, a file is opened and read through each row. On the very first row read, we are given the header row. With the header row, we can build a header map and a column index. The header map connects column name to index whereas the column index allows for faster parsing. The TaxiTripParser class contains methods to convert the data in a row to their corresponding primitive datatype. A TaxiTripRecord class acts as the model for each row, being set accordingly after all conversions. From ingestion, performance is heavily affected by how the datetime for pickup and dropoff are represented. Since the original format is a string, these values are converted to their epoch UTC representations. This means every row has further computation to convert these two columns. TaxiTripRecord holds a vector of TaxiTripRecord objects.

2) *Failed Datetime Parsing*: A first attempt at converting the datetime format involved using stringstream, however, this was much slower than manual parsing with a time_t conversion. This was because stringstream introduces a lot of overhead when parsing to be flexible. With the dataset being large, the time taken to parse the datetime string would cost performance. Since the datetime was a consistent format, it could be manually parsed instead and converted to time_t at a much faster rate.

3) *Query System*: The TaxiTripQuerySpec class defines optional constraints to implement a predicate-based API. Here, the following rows are focused on: pickup/dropoff datetime, trip distance, total amount, tip amount, and payment type. We also introduce two query types: count and execute. Count queries

simply return a count of how many records are in the query subset whereas `execute` returns references.

4) *Abstraction and Facade Design*: To abstract all of the classes with a facade design pattern, the benchmark runner combines all the classes together. The benchmark runner will start the pipeline of ingestion, parsing, and querying a specified number of execution times by the user. Time to ingest and time to query is recorded along with rows read, read failures, rows scanned, and matches. Total data scanned, row throughput, IO throughput, parse failure rate, and selectivity are also derived metrics.

5) *Benchmark Metrics*: For each run, the following raw metrics are recorded: ingestion time, query time, rows read, parse failures, rows scanned, matches returned. From these values, derived metrics are computed: total data scanned, row throughput, I/O throughput, parse failure rate, and selectivity. Ten runs are executed to compute average performance metrics and reduce variability.

6) *Performance Observations*: Ingestion is dominated by disk I/O, CSV parsing, and datetime conversion. The system must fully parse all CSVs before allowing queries to be done. The average ingestion time across 10 runs was **2505.58 seconds** with row ingestion throughput at an average of **23.30 million rows per second**. Parse failure rate was at a constant 0.00636944% but this may be an incorrect assumption of what a parse failure is on the dataset as we check if any row is negative. Further data retrieved is shown in Table I.

TABLE I
PHASE ONE IN-MEMORY BENCHMARK DATA

Run	Ingest	Count	Exec	Rows (x 10 ⁷)	IO (MiB/s)
1	2498s	7.91s	7.47s	2.27	1732
2	2495s	9.95s	7.52s	1.8	1377
3	2496s	7.96s	7.39s	2.26	1721
4	2495s	7.98s	7.37s	2.25	1717
5	2494s	9.79s	7.42s	1.84	1400
6	2495s	7.92s	7.41s	2.27	1730
7	2519s	6.83s	7.61s	2.63	2008
8	2520s	6.75s	7.54s	2.66	2029

9	2526s	6.87s	7.55s	2.61	1994
10	2519	6.62	7.49	2.71	2068

B. Streaming

1) *Streaming Architecture*: We chose a streaming design instead of in-memory loading to process multi-GB CSV datasets while keeping memory bounded. **TaxiTripDataset** scans the input directory and streams each CSV file line-by-line using `BufferedReader`. Each line is parsed by `TaxiTripCSVParser` into a reusable `TaxiTrip` record. The system performs a full sequential scan where each row is read, parsed, and evaluated against a predicate. Results are aggregated into counters without storing raw records, ensuring constant memory usage independent of dataset size.

2) *Query and Benchmarking*: Queries are expressed through predicate classes implementing `ITripPredicate`. `PickupTimeRangePredicate` and `FieldRangePredicate` support range filtering, while complex filters are composed using `AndPredicate`. A `TruePredicate` is used for match-all baseline runs. `QueryResult` aggregates statistics across files, and the benchmark runner measures execution time, throughput (rows/sec and MiB/sec), and correctness metrics such as rows scanned, matched, and parse failures.

3) *Performance and Interpretation*: Experiments were conducted on a multi-core macOS system with 16 GB RAM using a ~12 GB dataset distributed across more than 30 CSV files. The serial baseline (match-all predicate) was executed 10 times, and the results are summarized in **Table II**. As shown in **Table II**, the Phase 1 serial baseline completed 10 runs with an average execution time of **841.66 seconds**(min 825.79 s, max 856.85 s). Average throughput was approximately **129,825 rows/sec (12.55 MiB/sec)** over 109,247,536 rows, with zero parse failures. A secondary 5-run verification (avg $\approx$ 846 s) produced similar results, confirming consistency.

The streaming implementation demonstrated stable performance across runs, with minimal variance. CPU utilization remained consistently high, indicating that execution is dominated by per-row parsing and processing rather than disk idle time. Memory usage remained low and stable, validating the bounded-memory streaming design. The pickup-time range query completed in comparable time to the match-all baseline and correctly filtered trips within the specified window. Since all rows must be read and parsed regardless of selectivity, runtime

is governed by full dataset traversal and parsing costs rather than predicate complexity. Overall, the results in **Table II** establish a reliable serial baseline for evaluating parallel scalability in Phase 2.

Table II
PHASE I BASELINE PERFORMANCE- STREAMING

Query(Run s)	Avg Time(s)	Rows/s	MiB/s
Matchall(10)	841.66	129,825	12.55
Pickuptime range(10)	855.00	127,800	12.40
match-all	846.17	129,242	12.50

C. Analysis

Phase One evaluates two serial designs: an in-memory ingestion model and a streaming execution model. In the in-memory approach, all CSV files are fully parsed and loaded into structured TaxiTripRecord objects before queries execute. Across 10 runs, average ingestion time was approximately **2505.58 seconds**, with a parse failure rate of **~0.006%**. Performance was dominated by disk I/O and per-row parsing, particularly datetime conversion from string to epoch format. Optimizing datetime parsing significantly improved ingestion efficiency, highlighting parsing as a key cost component. This model supports flexible post-ingestion queries but requires memory proportional to dataset size.

The streaming implementation processes files line-by-line and evaluates predicates immediately without storing records. The serial baseline (match-all) averaged **841.66 seconds** over 10 runs (min 825.79 s, max 856.85 s), scanning **109,247,536 rows** at approximately **129,825 rows/sec (12.55 MiB/sec)** with zero parse failures. Because all rows must be parsed regardless of selectivity, runtime is governed by full dataset traversal and parsing cost. Together, these serial implementations establish a stable functional baseline for evaluating parallel scalability in Phase Two.

IV. PHASE TWO - PARALLEL

Using OpenMP threading, parallelization is introduced into the application. The goal of this phase was not to change the data model, but to improve performance by leveraging

multi-core processing while keeping the underlying architecture consistent with Phase One. The parallelization was applied to two areas: ingestion and querying.

A. In-Memory

1) *Parallel Ingestion*: In phase one, ingestion was purely serial. A CSVReader iterated through all files in the data directory, and each row was parsed and appended into a vector of TaxiTripRecord objects. While this approach accomplished the project's goals it fully serialized file traversal and parsing. In phase two, ingestion was parallelized at the file level using OpenMP. Since the dataset consisted of multiple yearly CSV files, this allowed safe parallelization without splitting individual files or handling line-boundary issues. Each thread was assigned one file at a time using an OpenMP parallel-for construct over the file list. Since each file was parsed independently, no coordination was required during parsing. The only sequential section occurred during merging the file data together but this was inexpensive relative to total parsing time. This model scales with the number of files, providing a safe ingestion parallelization strategy. In benchmarking, the average ingestion time decreased from **2505.58 seconds** to **1200.57 seconds**, yielding an approximate speedup of **2.09x**. The speedup is bounded by the number of CSV files available, disk I/O bandwidth, and datetime parsing cost. Since ingestion involves disk I/O and CPU-bound parsing, speedup was not linear with thread count. Figure 1 shows the ingestion time comparison with phase one.

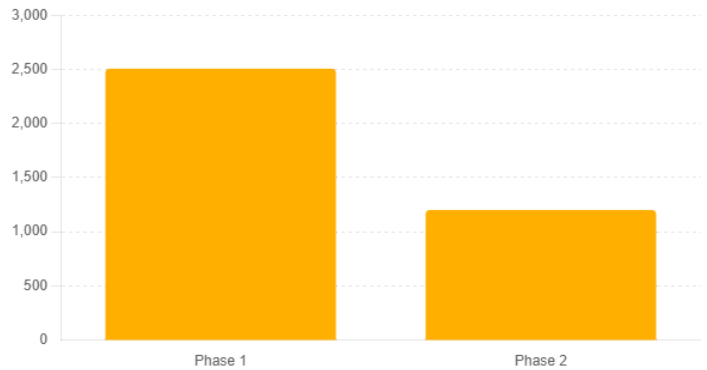


Fig. 1. Ingestion performance comparison between phase 1 and phase 2. Phase 2 yields a speedup of 2.09x.

2) *Parallel Query*: In phase two, query execution was parallelized using OpenMP. The dataset was divided into contiguous index regions, with each thread responsible for

evaluation of a chunk of rows. For the count query, each thread processed a subset of indices while a thread-local counter accumulated the number of matches. Then, OpenMP reduced all thread-local counters into a final total. For the execute query, a two-pass strategy was used to avoid locking during push operations. Each thread counted the number of matches in its region and a prefix sum over thread-local counts determined output offsets. Threads wrote matching row references into their pre-allocated output regions.

3) *Performance Observations*: Parallel row throughput was an average of **17.12 million rows per second**, resulting in a negative speedup for queries. This is further shown when the average time for a query in phase one took **7.86 seconds** compared to phase two's **10.59 seconds**. This was most likely due to resource contention as the workload was memory-bandwidth bound, resulting in threads processing fewer rows per second. Further data from benchmark execution is shown in Table III.

TABLE III
PHASE TWO IN-MEMORY BENCHMARK DATA

Run	Ingest	Count	Exec	Rows (x 10 ⁷)	IO (MiB/s)
1	1226s	10.77s	6.67s	1.67	1272
2	1180s	12.02s	5.92s	1.49	1140
3	1179s	10.03s	4.38s	1.79	1366
4	1182s	9.56s	4.32s	1.88	1434
5	1281s	10.06s	4.37s	1.79	1362
6	1193s	10.75s	4.42s	1.67	1275
7	1188s	9.01s	4.39s	1.99	1521
8	1193s	11.53s	4.27s	1.56	1189
9	1184s	9.83s	4.29s	1.83	1394
10	1201s	12.35s	4.27s	1.45	1110

B. Streaming

1) *File-Level OpenMP Design* : Phase Two extends the streaming pipeline using file-level parallelism with OpenMP. Since the dataset contains multiple CSV files, different files are processed concurrently while preserving sequential row parsing within each file. The TaxiTripDataset class was extended with queryDirParallel and queryFilesParallel, which distribute files

across threads using `#pragma omp parallel for schedule(dynamic)`. Each thread invokes the existing queryFileroutine with thread-local instances of TaxiTripCSVParser, TaxiTrip, and a reusable line buffer, maintaining the Phase One streaming logic. Aggregate counters (rows_scanned, rows_matched, rows_parse_failed) are combined using OpenMP reduction to avoid explicit locking. Experiments were conducted on macOS using the libomp runtime on a multi-core system with 16 GB RAM.

2) *Measured Scaling Behavior*: Performance was evaluated using **1, 2, 4, and 8 threads**, and results are summarized in Table IV. Instead of improving performance, runtime increased as thread count increased. At 2 threads, execution time rose to approximately 1840–1945 seconds (about 120–130% slower than the serial baseline). At 4 threads, runtime increased further to approximately 2569 seconds (about 200% slower than serial), and at 8 threads it increased to 2721.78 seconds, corresponding to a speedup of only 0.309× and an efficiency of 3.9%.

Throughput declined from approximately 131,000 rows/sec (1 thread) to about 59,000 rows/sec with 2 threads, about 42,500 rows/sec with 4 threads, and 40,138 rows/sec with 8 threads, demonstrating clear negative scaling.

3) *Compute Bound Execution and Scaling Failure*: CPU utilization during parallel runs remained high (≈ 390 –450%), indicating active multi-core execution rather than I/O blocking. Profiling showed that roughly 94% of total execution time was spent in CSV parsing and predicate evaluation, while only about 6% was spent on line reading. This behaviour is confirmed in Fig 2, where throughput decreases steadily as thread count increases. Since parsing dominates execution, adding threads increases concurrent parsing work without reducing per-row computation. The negative scaling observed in Table IV is therefore attributable to CPU-side contention in parsing and processing rather than disk I/O limitations. Performance could potentially improve only if the parsing stage itself were optimized or restructured—for example, by reducing string conversion overhead, simplifying datetime parsing, or introducing finer-grained parallelism within large files—rather than relying solely on file-level concurrency.

C. Analysis :

Phase Two introduced file-level OpenMP parallelization for both the in-memory and streaming implementations, yielding different outcomes. In the in-memory model, parallel ingestion

across independent CSV files reduced average ingestion time from **2505.58 seconds to 1200.57 seconds**, demonstrating measurable improvement when work could be partitioned cleanly with minimal synchronization. This confirms that file-level parallelism is effective when tasks are independent and merge overhead is small relative to parsing cost.

In contrast, the streaming implementation exhibited negative scaling. Compared to the serial baseline (~846 seconds), execution time increased to approximately **1892 seconds (2 threads)**, **2569 seconds (4 threads)**, and **2722 seconds (8 threads)**, with corresponding declines in throughput as thread count increased. Because streaming requires full parsing and predicate evaluation for every row, concurrent execution introduces additional overhead without reducing per-row computational cost. Overall, Phase two shows that parallelization depends on workload structure: file level ingestion scaled, whereas streaming remained constrained by per-row parsing.

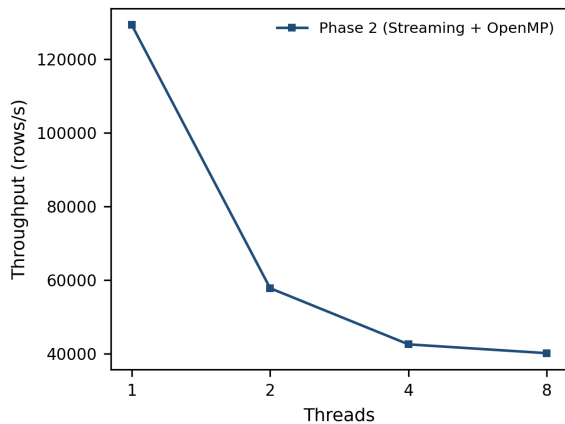


Fig2. Throughput degradation under file-level parallelism in the streaming implementation

TABLE II
PHASE II PARELLEL PERFORMANCE-STREAMING

Threads	Time(s)	Rows/s	MiB/s	Eff.(%)
2	1892.14	57,782	5.59	22
4	2421.00	45,000	4.11	9
8	2721.78	40,138	3.88	3.9

V. PHASE THREE - MODEL REDEFINITION

The final phase has the record store be redefined into a vectorized organization, along with any code updated and enhanced for this optimization.

A. In-Memory

1) *Object-of-Arrays (SoA) Redesign*: Previously, each TaxiTripRecord contained all columns. During queries, if only a subset of columns was required, the CPU loaded entire records into the cache. In phase three, the store was redefined as having each column stored as its own vector. This design reduced memory overhead as executed queries returned row indices instead of object references.

2) *Parallel Ingestion with SoA*: Ingestion remained relatively the same but we avoided constructing large temporary TaxiTripRecord vectors and allowed direct column-wise append operations. The merge step appended column vectors instead of individual record objects, reducing overhead.

3) *Parallel Query Over SoA*: Parallelization became more efficient as each thread processed contiguous index regions, only necessary columns were accessed, and fewer cache lines touched. The query operations remained the same to phase two with adjustments to support the new model definition. Previously, estimated bytes scanned was **13703MiB** whereas under SoA, only predicate-relevant columns were scanned, approximating **4453.47MiB** scanned, highlighted in Figure 3. This reduced memory bandwidth requirements and improved throughput from **17.12 million rows per second to 213.06 million rows per second**.

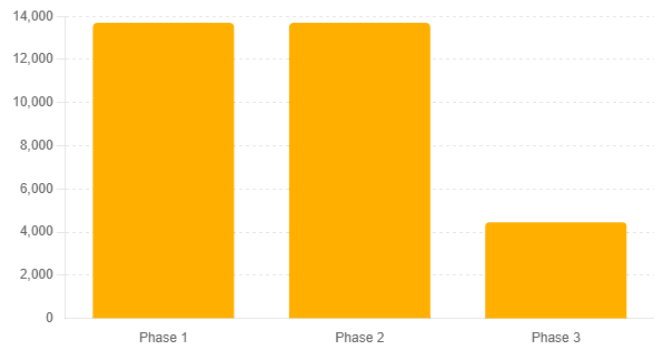


Fig. 3. Data Scanned Per Query Over Phases. Phase 3 only uses necessary column vectors.

4) *Performance Improvements*: Overall query time significantly improved from an average of **10.59 seconds** to

0.84 seconds, resulting in a speedup of **12.61x**. Ingestion time still remained partially I/O bound and datetime-parsing bound whereas query performance increased as we only used necessary column vectors rather than entire objects. The performance data of phase three is noted in Table V.

TABLE V
PHASE THREE IN-MEMORY BENCHMARK DATA

Run	Ingest	Count	Exec	Rows (x 10 ⁸)	IO (MiB/s)
1	1180s	0.837s	1.581s	2.15	5321
2	1183s	0.846s	1.564s	2.12	5266
3	1183s	0.837s	1.59s	2.15	5319
4	1185s	0.838s	1.568s	2.14	5314
5	1178s	0.844s	1.569s	2.13	5274
6	1183s	0.844s	1.568s	2.13	5276
7	1181s	0.847s	1.593s	2.12	5260
8	1186s	0.851s	1.577s	2.11	5232
9	1185s	0.85s	1.579s	2.11	5241
10	1177s	0.838s	1.57s	2.14	5314

B. Streaming

Phase Three was not extended to the streaming implementation following Phase Two results. Parallel streaming exhibited negative scaling, indicating that execution time was dominated by full traversal and per-row parsing costs. The SoA redesign in Phase Three restructures in-memory storage to improve data layout and access patterns; applying it to streaming would have required a fundamental architectural change rather than an incremental optimization. Therefore, Phase Three focused on the in-memory model, where data layout directly influenced observed performance.

VI. CONCLUSION

This study evaluated two architectural approaches, in-memory storage and streaming execution, across three development phases. The results demonstrate that architectural design, memory layout, and workload characteristics significantly influence performance.

TABLE VI
SUMMARIZED IN-MEMORY BENCHMARK DATA

Phase	Ingest	Count	Exec	Rows/s	IO (MiB/s)	Data (MiB)
1	2506s	7.86s	7.48s	2.33×10^7	1778	13703
2	1201s	10.59s	4.73s	1.71×10^7	1306	13703
3	1182s	0.843s	1.576s	2.13×10^8	5282	4453

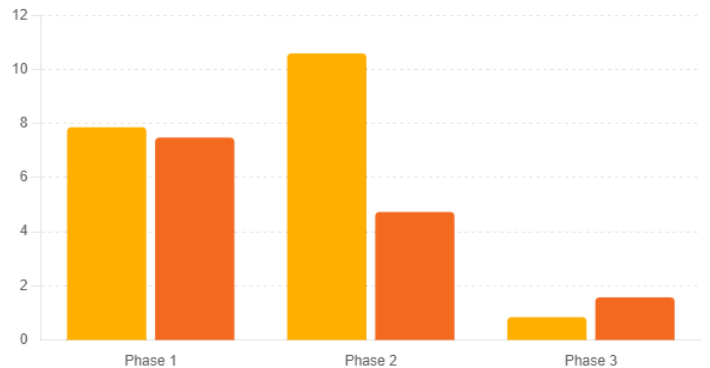


Fig. 4. Query execution comparison for in-memory approach.

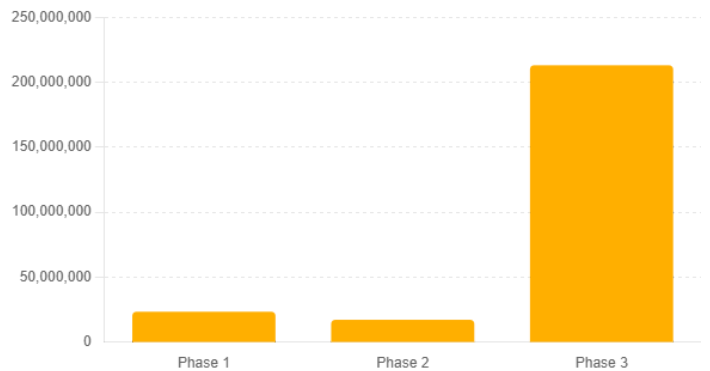


Fig. 5. Row throughput comparison for in-memory approach.

1) *Phase One Serial Baseline*: In phase one, both implementations were fully serial. The in-memory approach required a substantial upfront ingestion cost, averaging **2506 seconds** to load and parse the dataset. However, once loaded, query execution was fast. The average query count time was **7.86 seconds**, the average execute query time was **7.48 seconds**, and the row throughput was **2.33×10^7 rows/sec**. In contrast, the streaming approach avoided the large upfront ingestion time. Since records were not stored in-memory, total execution time

averaged approximately under **855 seconds** for a pickup time range query. Throughput averaged **129,825 rows/sec**. While streaming eliminated the ingestion delay, it required full dataset traversal for every query. Consequently, for repeated queries, total execution time significantly exceeded that of the in-memory model. Thus, phase one revealed a fundamental tradeoff of having high upfront cost for faster queries or no upfront cost for slower query execution.

2) *Phase Two Parallelization*: Parallel ingestion across independent CSV files reduced average ingestion time from **2505.58 seconds** to **1200.57 seconds**, achieving a **2.09x speedup**. This confirmed that file-level parallelism is effective when work is independent and merge overhead is small. However, query parallelization yielded negative scaling. Phase one's count query of **7.86 seconds** increased to **10.59 seconds** on average. Furthermore, the row throughput decreased from **2.33×10^7 rows/sec** to **1.71×10^7 rows/sec** and the I/O throughput decreased from **1778 MiB/s** to **1306 MiB/s**. The degradation indicates that workload became memory-bandwidth bound. Multiple threads competed for shared memory resources, resulting in diminished per-thread performance. Since the data layout remained as an Array-of-Structures (AoS), each thread still loaded entire records into cache even if only a subset of fields were needed. Parallel streaming exhibited even stronger negative scaling. With its baseline at **846 seconds**, two threads resulted in **1892 seconds**, four threads at **2569 seconds**, and 8 threads at **2722 seconds**. Throughput decreased as thread count increased, confirming that streaming was dominated by per-row parsing rather than I/O wait. Since approximately 94% of execution time was spent parsing and evaluating predicates, parallelism increased contention without reducing computational complexity. Phase two demonstrated that parallel ingestion benefits when tasks are independent, a parallel query does not guarantee speedup when memory bandwidth is the bottleneck, and streaming parallelization suffers when dominated by CPU-bound parsing.

3) *Phase Three Model Redefinition*: Phase three restructured the in-memory model from an Array-of-Structures (AoS) layout to an Object-of-Arrays (SoA) layout. Previously, each record stored all columns contiguously, causing unnecessary memory loads during queries. Under SoA, each column was stored in its own vector, and queries accessed only the necessary columns. This significantly reduced memory traffic with phase one and two scanning an estimate of **13,703**

MiB of data versus phase three's predicate-relevant scanned data of **4,453 MiB**. We see an approximate 3.1x **reduction in memory scanned per query**. Performance improvements were also substantial with average count time reduced from **10.59 seconds** to **0.843 seconds**, execute time reduced from **4.73 seconds** to **1.576 seconds**, row throughput increased from **1.71×10^7 rows/sec** to **2.13×10^8 rows/sec**, and I/O throughput increased from **1306 MiB/s** to **5282 MiB/s**. Overall, the count query's relative speedup in comparison with phase two was **12.61x**. Notably, ingestion time remained approximately 1182 seconds, indicating that ingestion was still partially I/O-bound and dominated by datetime parsing. The major gains were realized in query execution due to improved cache locality and reduced memory bandwidth pressure.

Across all phases, the following conclusions emerge:

- 1) Streaming is efficient for one-time scans but does not benefit from repeated queries.
- 2) In-memory execution is initially expensive but provides dramatically faster query performance once data is loaded.
- 3) Parallelization alone does not guarantee speedup, especially in memory-bound workloads.
- 4) Memory layout has a larger impact on performance than threading alone.

Ultimately, this report illustrates that for large-scale data processing, memory layout and data access patterns often dominate performance more than parallelization alone. This reinforces the importance of aligning system architecture with workload characteristics when optimizing high-volume data systems.